CODEBASE REVIEW & PHASED ROADMAP · REVISION 2

Autonomous AI Analyst:

what shipped, and what comes next

Revision 1 found a live API key in a public repo, six silent correctness bugs, and a model presented as *optimized* at 1.7% accuracy. **All seven phases of that plan are now implemented.** This revision records what was actually built, states plainly what was *not*, and sets the next three phases — which are no longer about fixing the code, but about proving it holds up outside a laptop.

REVISED 18 Aug 2026 **SUPERSEDES** Rev 1 · 16 Aug 2026 **SCOPE** backend/ + frontend/, 92 source files

STACK FastAPI · scikit-learn · XGBoost · LightGBM · SHAP · React 18 + Vite · Gemini

The same four numbers, two days later

Revision 1 opened with these. They are the honest measure of whether the plan was worth following.

BLOCKING

0 ← 1

No credential is reachable from the tree. One operator action remains outstanding — see *Carried forward*.

CORRECTNESS BUGS

0 ← 6

Leakage, SHAP misalignment, target detection, label encoding, cache keying, and the unreachable chat path — each closed with a test that fails without the fix.

TEST COVERAGE

412 ← 0

333 backend and 79 frontend tests across 25 files, plus ruff, gitleaks, and CI on every push. Dependencies fully pinned.

SHIPPED ACCURACY

Refused

Was 1.7%, presented as the Optimized Model. The dataset that produced Exhibit A is now rejected at target selection, before a model is ever fitted.

Exhibit A, revisited

The 19,245-row healthcare run remains the clearest test of whether anything changed. Feeding the same CSV through the current pipeline, four independent guards now fire before a misleading number can be produced:

```
1. Target validation   "Findings" has 14,555 unique values across 19,245 rows.
                       Rejected as free text, not a label — with a column picker
                       offered instead of a silent last-column guess.
2. Cardinality ceiling One-hot encoding refuses to expand a column past its
                       limit, so 14,555 phantom features cannot be created.
3. Label encoding      Classification targets are encoded and the encoder is
                       stored with the model, so XGBoost no longer fails on strings.
4. Health verdict      A run that scores near chance is labelled as such. Nothing
                       is called "Optimized Model" on the strength of 1.7%.
```

The theme of Revision 1 was that the failure modes were silent. The theme of this revision is that they are now loud, and that the loudness is under test.

What shipped, by phase

Each phase measured against the exit criteria it was given in Revision 1, not against effort spent.

PH	THEME	STATUS	DELIVERED AGAINST ITS EXIT CRITERIA
0	Stop the bleeding	SHIPPED	Hash validation on every path-taking route, streamed uploads under byte/row/type ceilings, explicit CORS origins, gitleaks in pre-commit and CI. *Key rotation and history purge remain the operator's to do.*
1	Trust the numbers	SHIPPED	Split-before-fit inside a single `Pipeline`; SHAP names carried through `get_feature_names_out()` and aggregated to parent features; target and task detection rebuilt with a user override; label encoding stored with the model; cache keyed on the full configuration.
2	Foundation	SHIPPED	Tests placed where the bugs were, `pydantic-settings`, structured JSON logs carrying request and job ids, jobs in SQLite via SQLModel, pinned requirements compiled with `uv`, Dockerfiles, compose, and CI that fails on lint, tests, build, or secrets.
3	Close the loop	SHIPPED	Charts, correlation heatmap, confusion matrix and residual plots rendered; a dataset workspace with reopen, compare and delete; a per-column profile ahead of training; prediction results as a table with probabilities; honest parse reporting; polling with abort and cleanup.
4	ML depth	SHIPPED	Stratified cross-validation with mean ± std over an untouched holdout, imbalance-aware metrics and selection, class weighting and optional SMOTE, a time-budgeted tuner, voting ensembles, a broader registry including LightGBM, and a model card per run.
5	The analyst	SHIPPED	A server-side sandboxed interpreter with an audit hook and resource limits — replacing the Files-API upload, so rows no longer leave the machine; explicit tools over free-form code; a provider interface testable with a fake; per-conversation guards; one-button report generation.
6	Multi-user	SHIPPED	Detailed below — it is the newest work and the least exercised.

Phase 6 in detail

Every capability is opt-in and defaults to off, because the first five phases were built for one trusted operator on localhost and turning any of this on by default would break that install to solve a problem it does not have.

CAPABILITY	SWITCH	DECISION WORTH RECORDING
Auth & isolation	`AUTH_ENABLED=false`	Bearer API keys stored only as SHA-256 and shown once. Someone else's run answers **404, not 403** — a run key is a content hash, so confirming it exists leaks both the dataset and its holder.
Cache sharing	—	Rev 1 asked whether cross-user artifact sharing is a feature or a leak. Answered **leak**: the owner is part of the cache key, so identical files are stored twice rather than shared.
Job queue	`QUEUE_BACKEND=inline`	RQ over Redis with a separate worker; the API no longer fails in-flight jobs on restart. An unreachable Redis **falls back to inline** rather than accepting jobs nothing would run.
Object storage	`STORAGE_BACKEND=local`	Artifacts mirrored to S3 after a run and pulled back on a miss. Credentials come from the boto3 chain, never from settings, so they cannot reach a log line.
Observability	`METRICS_ENABLED=true`	Prometheus with **bounded labels** — run keys collapse to `:id`, so the endpoint cannot become a memory leak. Sentry with PII off. LLM cost is always labelled an estimate.
Data lifecycle	`RETENTION_DAYS=0`	Owner and expiry on every run; deletion removes bytes, not just rows; `GET /api/privacy` returns the policy as data so docs and UI cannot drift.
Artifact integrity	always on	Pickle was **signed, not replaced** — no portable non-pickle format exists for sklearn Pipelines. HMAC-SHA256 verified *before* `joblib.load`, never after.

Carried forward

What a status report is for. These are open, and three of them are the reason the next phase exists.

ITEM	SEV	PH	WHY IT IS STILL OPEN
Gemini key not rotated	CRIT	—	The single most important line of Revision 1, and the one item no amount of code can close. The key was public; it must be revoked in Google AI Studio and reissued. **Owner action, still outstanding.**
`.env` **still in git history**	CRIT	—	Untracked from HEAD, but `git filter-repo` plus a force-push has not been run. Cleanup rather than remediation — rotation is what protects you — but it should not be left indefinitely.
Queue path never run end to end	HIGH	7	The RQ worker, Redis service and handover are implemented and unit-tested — the job reaches the queue, its input lands on shared storage, and the arguments survive the pickle round-trip RQ performs. But no real worker has consumed a real job: the Docker daemon was unavailable. **Verified by construction, not by observation.**
SQLite under multiple processes	HIGH	7	The moment the queue is enabled, an API process and a worker write the same file concurrently. SQLite serialises writers with a lock; under load that surfaces as `database is locked`, not as a clean error. Postgres is the answer and the code already accepts a URL.
No migration framework	HIGH	7	`_add_missing_columns` adds nullable columns to existing tables and is honest in its own docstring about being narrow. Anything structural — a rename, a constraint, a backfill — needs Alembic.
Rate limiting is per conversation	MED	8	Guards slow one session; they do not stop a client opening a hundred. With authentication now present there is finally a principal to attach a real limit to.
No key rotation, expiry, or audit log	MED	8	A key can be issued and deactivated, and that is all. There is no record of who deleted a dataset or created an account.
Sandbox is not a security boundary	MED	8	Documented as such. Sound against generated code that loops or phones home; not against a determined attacker. If untrusted users ever reach it, it needs a container with no network namespace and a seccomp profile.
743 kB frontend bundle	MED	10	One chunk, flagged by the build. Charting and the dashboard are the obvious split points.

THE HONEST SUMMARY

Phases 0–5 are exercised by 412 tests and have been run repeatedly. Phase 6 is exercised by 56 of them, but its two most consequential modes — a real Redis worker and a real S3 bucket — have only ever run against fakes. **Treat multi-user as implemented and unproven until Phase 7 closes.**

The next phases

Revision 1's phases were about a codebase that was wrong. These are about a codebase that is right on one machine and unproven on several. The sequencing rule is unchanged: each phase ships on its own, and 7 comes before 8 because there is no point hardening an edge that has never carried traffic.

PHASE 7 · 3–5 DAYS · DO THIS BEFORE TRUSTING PHASE 6

Prove the production path

Everything in Phase 6 is written and tested against substitutes. This phase replaces every substitute with the real thing and watches it work. It is deliberately small: the code exists, and what is missing is evidence.

- **Run the queue for real.** `docker compose --profile queue up`, upload a dataset, and `docker compose restart backend` while it trains. The run must complete and the job must report correctly afterwards. That is Revision 1's Phase 6 exit criterion, observed rather than argued.
- **Move to Postgres.** SQLite cannot be the store once a worker and one or more API processes write concurrently. The `DATABASE_URL` already accepts it; what is needed is a compose service, a connection-pool setting, and a run of the full suite against it.
- **Adopt Alembic** and generate the baseline from the current models, retiring `_add_missing_columns`. Every schema change after this gets a reviewable migration.
- **Exercise object storage against MinIO** — a compose service and the existing `S3_ENDPOINT_URL`. Train on one API replica, then serve a prediction from a second with an empty disk. That is the only assertion that proves the backend is genuinely shared.
- **Prove the artifacts are portable.** With more than one process verifying signatures, a generated-per-machine key silently breaks every cross-machine load. Set `ARTIFACT_SIGNING_KEY` explicitly and add a test that a bundle signed by one process verifies in another.
- **Back up and restore.** A documented drill: dump the database, snapshot the bucket, restore both into a clean stack, and confirm a pre-existing run still opens and still predicts.
- **Load test** the upload endpoint and one training run at expected concurrency, and record the numbers. Queue depth and training duration are already exported; this is what makes them mean something.

> **EXIT CRITERIA**
>
> A training run survives `restart backend` and is seen to finish; the full suite passes against Postgres with an Alembic-managed schema; a prediction is served from a replica that never trained the model; a restored backup produces a working system.

PHASE 8 · 1 WEEK · ONLY WITH UNTRUSTED USERS

Harden the edge

Phase 6 established *who* a caller is. This establishes what they may do, how much of it, and what is remembered afterwards. Skip it while every account belongs to someone you know.

- **Per-principal rate limiting** on the paid endpoints, replacing the per-conversation guard. A spend ceiling per account per day, enforced against the usage table that already records every call, so an abusive client is stopped rather than merely slowed.
- **Key lifecycle** — rotation without downtime, expiry, and more than one active key per account so a rotation is not an outage.
- **An audit log** of privileged and destructive actions: account creation and deactivation, dataset deletion, retention purges, key issuance. Append-only, and outside the retention policy it records.
- **Secrets out of the environment.** The signing key, the LLM key and the database URL should come from a secret manager once more than one person can read the deployment.
- **Make the sandbox a boundary** if untrusted users will reach it: a container with no network namespace, a seccomp profile, and a read-only filesystem — the step the current documentation explicitly defers.
- **Abuse-shaped tests.** The suite proves two users cannot see each other's data; it does not prove one user cannot exhaust the budget, fill the disk, or hold every worker.

> **EXIT CRITERIA**
>
> One client cannot exhaust another's budget or the machine's disk; a key can be rotated with no interruption and revoked with a stated blast radius; every destructive action is attributable to a principal and a time.

PHASE 9 · 1–2 WEEKS · THE GAP NOTHING HAS COVERED YET

Keep the model honest after it ships

Every phase so far judges a model at the moment it is trained. Nothing watches it afterwards. A model that was defensible in August and is quietly wrong by November is exactly the silent failure Revision 1 was written about — the same class of bug, moved from training time to serving time.

- **Log every prediction** — inputs, output, model version, and run key — so a served answer can be reproduced later. Under the same retention policy and the same owner scoping as everything else, because prediction inputs are as sensitive as the training rows.
- **Detect drift.** The per-column profile computed at training time is already the reference distribution; compare live inputs against it and raise a flag when a feature moves materially. The hard half is done, exactly as the chart payloads were in Revision 1's Phase 3.
- **Detect schema breakage** at serving time with a clear message — a renamed or missing column should say so, not produce a confident number from a silently imputed default.
- **Calibration** for classifiers whose probabilities are shown to users. A confidence figure the UI displays is a claim, and an uncalibrated one is a wrong claim.
- **Retraining triggers** — on drift, on a schedule, or on demand — reusing the queue from Phase 6, with a champion/challenger comparison so a retrained model is promoted on evidence rather than recency.
- **A monitoring view** per model: prediction volume, drift status, latency, and the model card. The model card already exists; this gives it a place to live after training day.

> **EXIT CRITERIA**
>
> A model whose input distribution has shifted raises a flag before a user reports a bad answer; any served prediction can be reproduced from its logged inputs and model version; a promoted model beat its predecessor on a metric you can point at.

PHASE 10 · OPEN-ENDED · PRODUCT, NOT ENGINEERING

Where it grows next

Nothing here is remediation, and none of it is sequenced. Listed so the roadmap does not end at "correct and observable", which is a floor rather than a destination.

- **Sharing** — a read-only link to a run or a report, scoped and expiring, now that ownership exists to scope it to.
- **Scheduled reports** — the Phase 5 report generator on a cron, delivered by email. The queue makes this nearly free.
- **Multiple datasets per question** — joins and comparisons across runs, which the analyst cannot express today.
- **Split the frontend bundle**, which the build has been warning about for two phases.
- **Export a trained pipeline** — ONNX where the preprocessing allows it, or a scoring container — so a model can leave the app it was trained in.

EXIT CRITERIA

None. These are product bets; judge them on use, not on completion.

In closing

If you only do one thing

Rotate the Gemini key. This was Revision 1's answer and it is still Revision 1's answer, because it has not been done. Every other item on this page is engineering; that one is a live credential that was published, and no amount of shipped code retires it.

If you only do one day

Start Docker, run `docker compose --profile queue up`, and restart the backend mid-training. Phase 6 is written, reviewed, and unit-tested, but its central promise — that a run survives the process that accepted it — has never been watched happening. One afternoon converts the whole phase from plausible to demonstrated.

Where the leverage is now

Phase 9. Revision 1 put the leverage in Phase 3 because the backend already computed charts nothing drew. The same shape has reappeared one layer up: training already produces a per-column profile, a model card, and a health verdict, and nothing consults any of them once the model is serving. Drift detection is mostly a comparison against data the app already stores.

What not to do

Do not turn on authentication, the queue, S3, and retention at once because they are available. Each defaults to off for a reason, each fails differently, and a stack where four subsystems changed at the same time is a stack where the next failure has four candidate causes. Turn them on one at a time, with the readiness endpoint in view.

What holds up

The judgement Revision 1 made about the architecture proved right: module boundaries stayed clean through seven phases of change, the async job pattern absorbed a real queue behind an interface without disturbing its callers, and content-addressed caching survived contact with multi-tenancy by taking one extra field. Roughly 5,000 lines were added across those phases without a structural rewrite. The layers needed work; the architecture did not.

> **STATUS OF THIS DOCUMENT**
>
> Revision 2 supersedes the 16 Aug 2026 review. Phases 0–6 are closed and recorded here for provenance; phases 7–10 are open. The findings ledger from Revision 1 is retained in git history at commit `7ad7ed7` — every finding in it is closed except the two operator actions listed under *Carried forward*.